

Buildify

version 1.0

Buildify is a modular geometry nodes library that helps with creating modular buildings based on a simple geometry.

GUIDE

Buildify can work in 2 modes.

Default [ADE](#) mode (art-directable editor) allows you more creative freedom, however you need to assign certain parameters manually. It's basically a normal edit mode, where you have to set up parameters and vertex groups by yourself.

Use [BLOSM](#) (Blender-OSM) mode to utilize addon's capabilities to write and override certain parameters for you, to quickly generate large scale areas using data from real world locations.

General

To begin with I would like to say that Buildify (BFY from now on) was made with the intent to be a modular geometry node system, that's why only a few parameters are exposed onto the modifier level. That prevents having dozens of parameters exposed. The system is built so the user can modify GN setup to fit their own needs and achieve a large variety of architectural styles.

Walls

Walls node group is the most basic module of building gen.



This node group creates a point cloud for instantiating modules from a selected collection. At least 2 inputs from the *Editor settings* node group are required (*base geometry* and *number of floors*).

Remove from bottom and *remove from top* parameters are used to delete floors from the bottom or top of the building which allows having different models for different building levels. By default this number is relative, but you can use math node in combination with *Number of floors* parameter to make more advanced setups (signs props node group is set up in the example file).

Flat pillar threshold determines the angle (in degrees) where flat pillars will be placed instead of concave/convex ones. This parameter is useful if you have slightly curved walls and you don't want to have pillars between every wall segment.

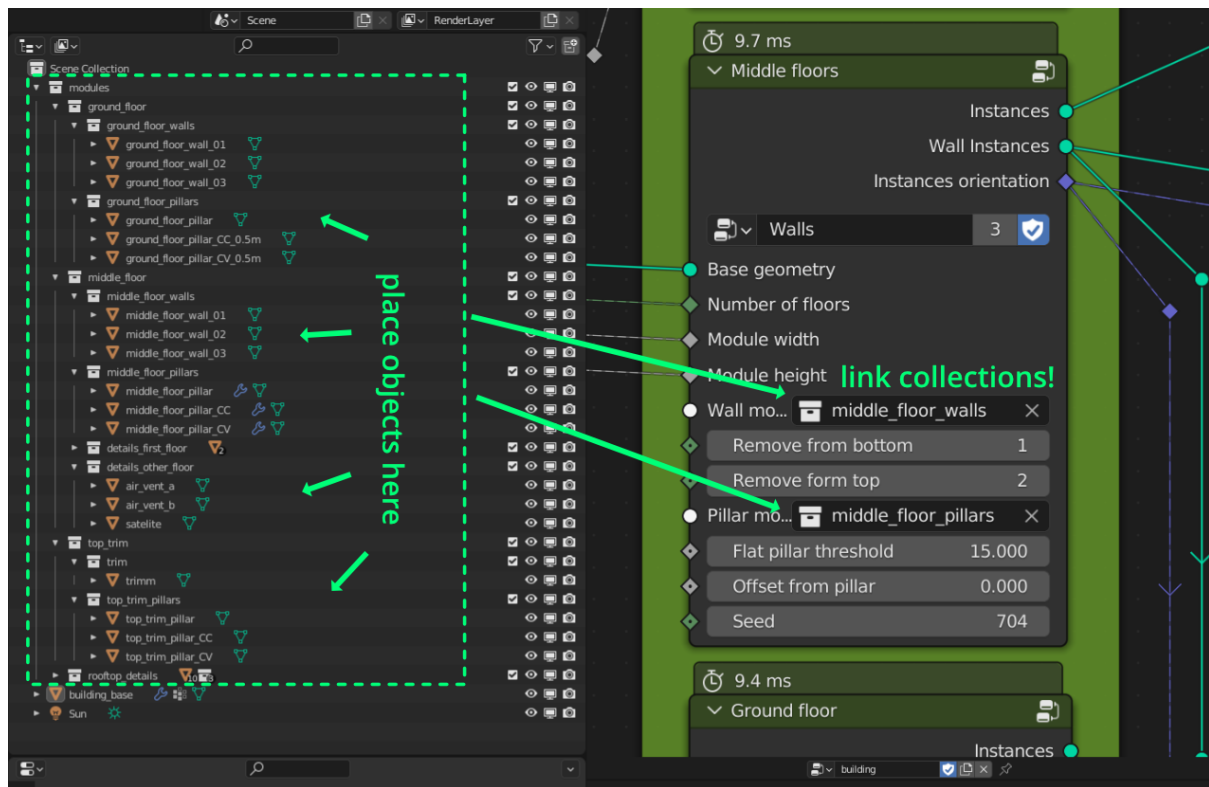
Module width and *module height* are dimensions of the used wall model from your building kit. If you have a 4x3m wall, you should set the model width to 4 and height to 3 otherwise the system won't work correctly. Currently more than 1 model dimension is not supported!

Buildify always tries to stretch the wall modules as best as possible to fit the shape of the building. However if the stretching is very noticeable, I suggest making an edge loop or subdividing the edge to get a more even distribution of modules.

If you are using Buildify for a game and you want to prevent model stretching, I suggest using multiples of grid units of your choice while using grid snapping.

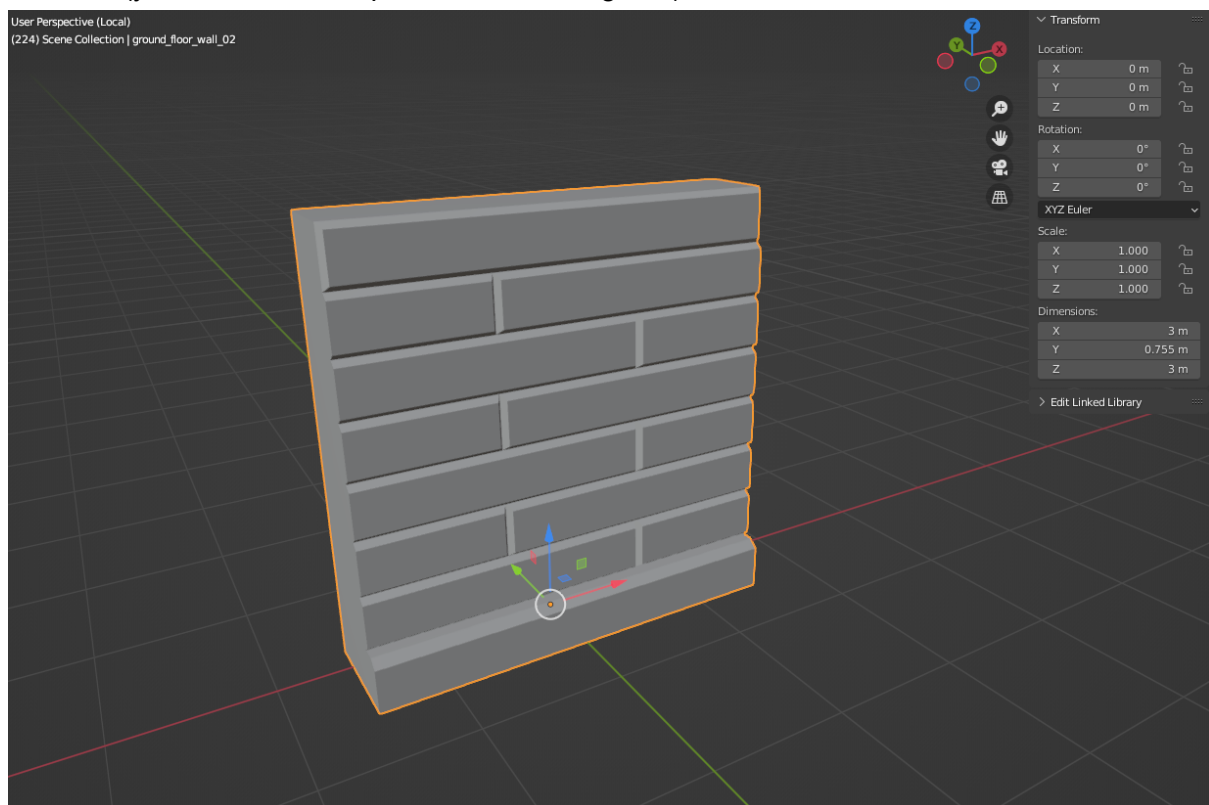
Replacing models

To be able to use your own models with buildify, just place models into corresponding collections in the outliner. In the example file there's one collection hierarchy made for you, but you can create and use your own. If you make your own collections hierarchy, make sure to link those collections into corresponding nodes in the node graph.



Collections for models replacement

Make sure that all models have applied rotation and scale (rotation set to $0^\circ, 0^\circ, 0^\circ$ and scale set to 1,1,1). You can apply these transforms by hovering with the mouse cursor over 3D view > ctrl+a > Apply rotation and scale. Also make sure that your model is facing the right direction (you can use example modules as a guide).

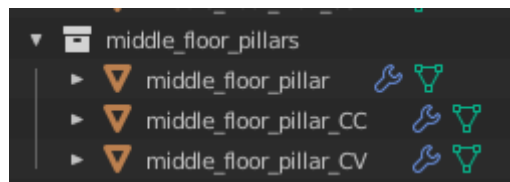


Applied scale and rotation on a module

Also make sure that the origin (pivot point) of your wall model is at the bottom of the wall, in the center horizontally.

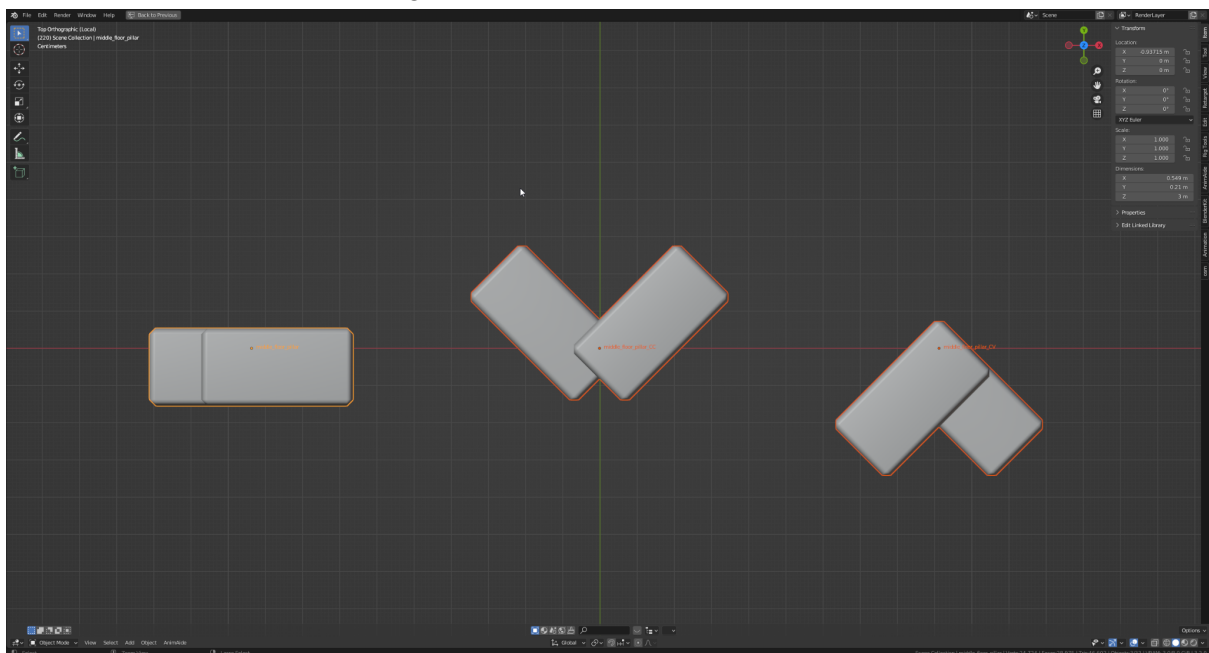
Pillars

To be sure that pillar models will be instantiated properly, make sure that pillars are inside one collection in following order: Flat pillar should be the first, followed by a pillar for concave corners and a pillar for convex corners goes last.



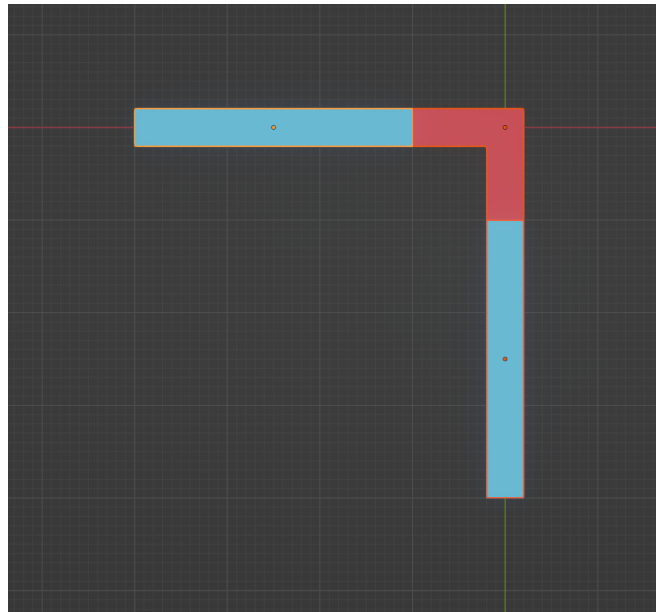
Proper pillar order

Also make sure that all pillars have applied (zeroed out) rotation and all of them are facing the positive Y axis (see the image below the view from top ortho camera).



Proper pillar orientation

Some building kits can have corner modules that do not protrude through the walls but rather connect with them in a seamless way. For that case I've added an *offset from pillar* parameter to prevent Z fighting. The parameter stretches walls away from the origin of the pillar, For the following example you should set *offset from pillar* value to 1 (1 meter).



Pillar offset

Flat roof and roof details

Currently the system supports only one roof type, the flat one.

The props (or detail objects) are placed on the roof in a really simple way, using recursive subdivision. Polygons are randomly subdivided several times and centers of those faces are grouped using ID node to instantiate models onto them.

Based on polygon size the node group chooses between predefined small, medium or large detail groups. These groups are already predefined, but you can add more groups if you dig deeper into the node group.

In the future I may extend this system even further to make some air condition systems and other more advanced logic, but some of them will require for loops which are not implemented in Blender yet.

Wall props

This node turns walls into a point cloud and further randomizes it and instantiates models onnto points.

Wall props node group requires at least 2 input from *walls* node group. These are *wall instances* and *instance orientation*.

Be creative!

Buildify currently does not support any more complex structure systems, so if you want to add more complex systems that should have some logic, I would suggest putting them inside one model (or a collection).

See the example below, how I've combined these 2 modules into one model with a pivot point in the middle (marked by red cross). This makes sure that each time this stair prefab is instantiated, it will always lead to a door instead to a wall. This model instance is then used only for the second floor, which makes sure to not instantiate stairs under the floor or into air.



Combine models in smart way to get more believable results

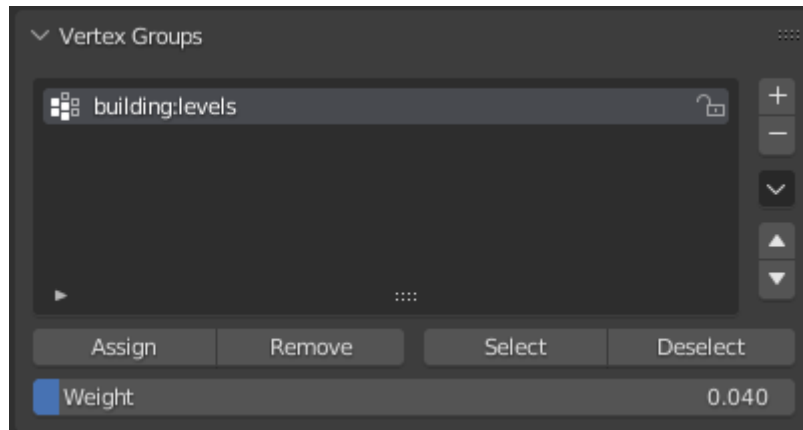
Another example would be to use big modules, taller than one floor such as towers, big sci fi modules and other unique decorative pieces. With such techniques you can make really complex looking shapes.

ADE

When working with ADE you can either let GN setup to generate the number of floors for each building randomly based on *Min number of floors* and *Max number of floors* parameters exposed to the GN modifier or you can set them manually for more art direction of your project.

Setting number of floors manually

Currently you can set up the number of floors for each building using vertex groups. Create a new vertex group called **building:levels** (this name must match otherwise it won't work). And then assign vertices to that vertex group. Weight determines the number of floors. Keep in mind that the **number is divided by 100 so for a building with 4 floor levels you have to assign vertices with weight 0.04**.



Assigning number of floors manually

It's a workaround until we will be able to assign integers or custom values onto vertices.

You have to assign each vertex of the building within the same vertex group weight otherwise the setup won't work correctly.

Multiple building styles

If you have multiple buildings inside one object (this might be caused by copy pasting base geometry in edit mode or using Blender-OSM addon) and you want to randomize their architecture styles, you can do it in 2 ways:

- a) destructively by selecting random buildings, separating them and applying multiple GN setups with different module kits. To select random buildings search for *Select random* and then Ctrl+L to make sure that all connected geometry is selected. Then press P and choose *selection*.

You can also separate buildings by loose parts by pressing P > by loose parts

- b) non-destructively by choosing a random building style inside one GN modifier. This will require a bit more work, but it offers better performance. On the example below you can see, how to make a setup inside one GN modifier that will randomly choose between (in this case) 3 different building styles.

KNOWN ISSUES AND LIMITATIONS

Weird results on tilted surface

To make the system work, you need to use a completely flat surface otherwise the system will be messed up. This is the limitation. However differences in elevation in disconnected geometry works as expected.

Flipped wall orientation when two or more edges on base geometry are overlapping

Actually this is something I don't know how to fix at the moment. The workaround is to offset these edges. Even 0.00001m translation helps.

FUTURE FEATURES

A list of features that I want to implement in future updates. The list does not go in any particular order.

More roof types

This is self explanatory.

Multiple building styles inside one object

Currently the system supports only one building type (architectural style) per object. I would like to enhance this behavior furthermore, so buildings can read attributes from BLOSM to generate buildings that fit different districts like (urban or suburban areas).

Customizable facades

A way to be able to set the front/side/back of the building to instance different modules, set gable and shape a roof according to it.

Support for flexible model sizes (6x3m, 8x3m, 3x6m...)

Currently the setup supports only one wall module size per building. So in the future I would like to extend this further to support flexible building modules. For example to be able to have big shop modules on ground floor or modules occupying multiple floors

Randomized props per building

This is mostly a feature specific to materials.
-shop brands, plaster color

FAQ

How can I add more roof types?

Currently only flat roofs are supported. I need to figure out how to properly instance models on different roof types as there's so many shapes and combinations and buildings can have different wall angles, not only right angle corners.

Lowest supported Blender version?

3.2.0, it won't work on older versions as some crucial nodes have been implemented in this version.

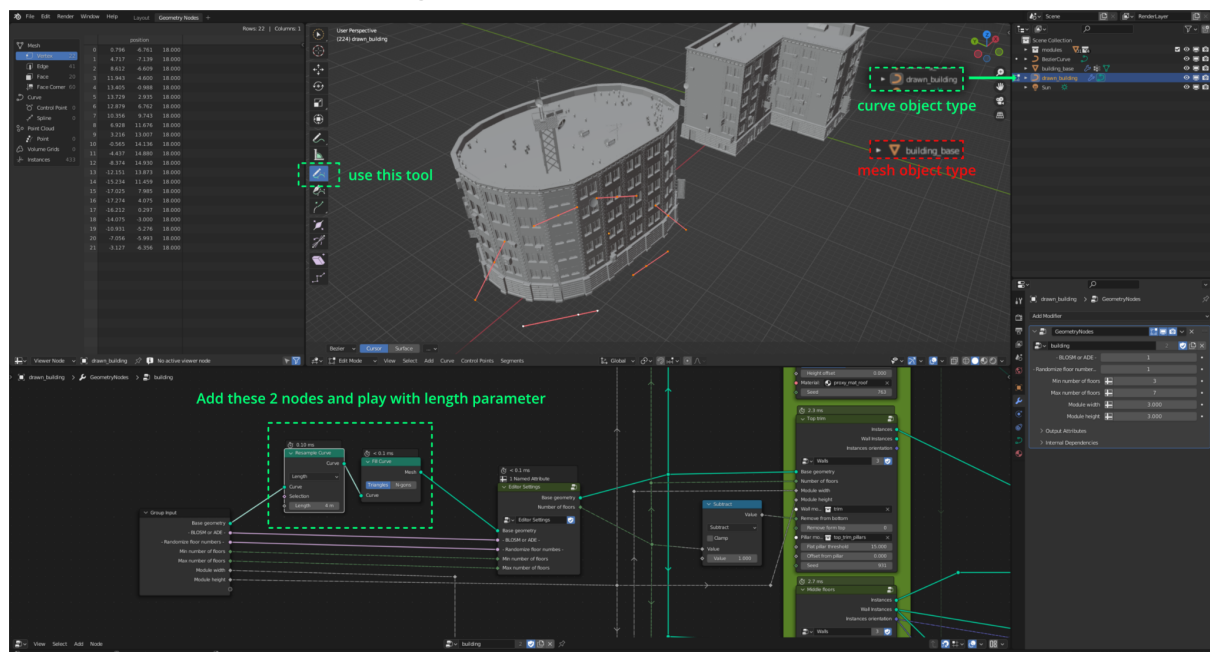
Where can I get modules from promotional materials?

Most of the modules come from Quixel Megascans library, so I am not able to share them within the tool. Instead the tool is provided with temporary greybox modules

Is it possible to prevent model stretching?

No. The system is designed that way, to be able to adapt any building shape. If you want to use the tool for a game for example ,where you don't want to have any model stretching, use grid snapping to fit the scale of your models.

How can I draw a building like in the trailer video?



Drawing mode for building creation

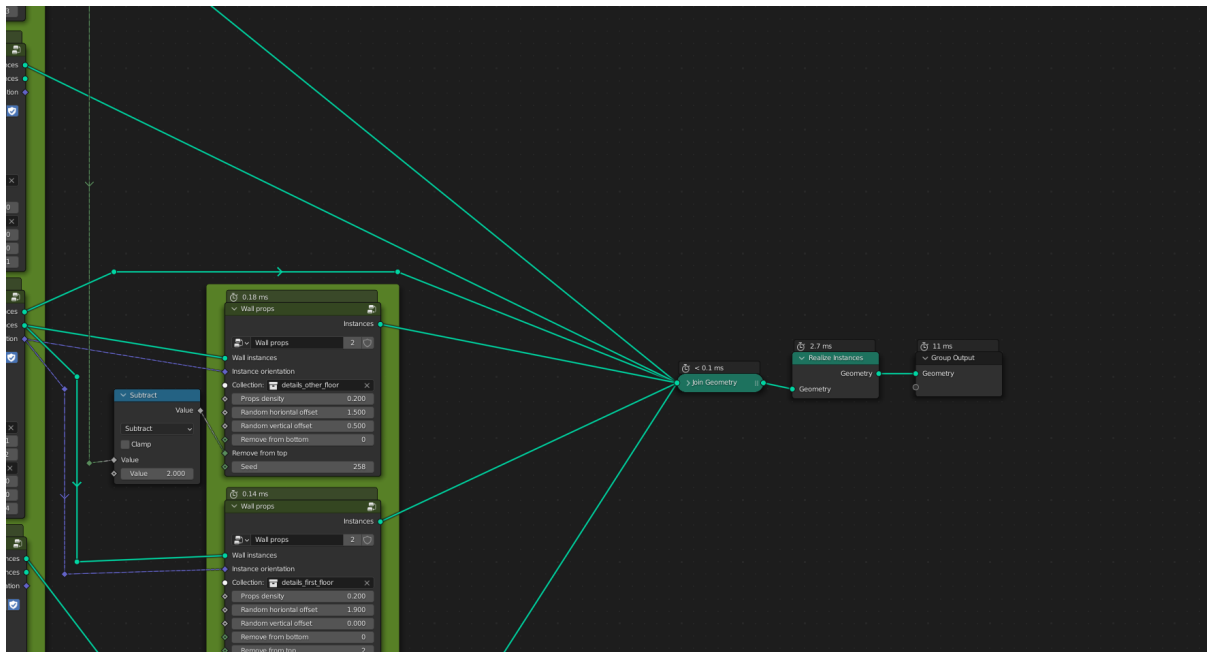
Create a curve object. Add a buildify modifier (you can copy it from existing object by selecting curve object, then the object with the modifier > ctrl+I > Copy modifiers. Add these 2 nodes into the geometry nodes tree. Use the Draw tool to draw buildings. Make sure that all strokes (control points) per building are in the same height otherwise you will get errors.

I've opened the .blend file, but there are only modules, the building is missing

Make sure to run Blender version 3.2. Older versions are not supported and 3.3 can be buggy sometimes (I am working on a solution).

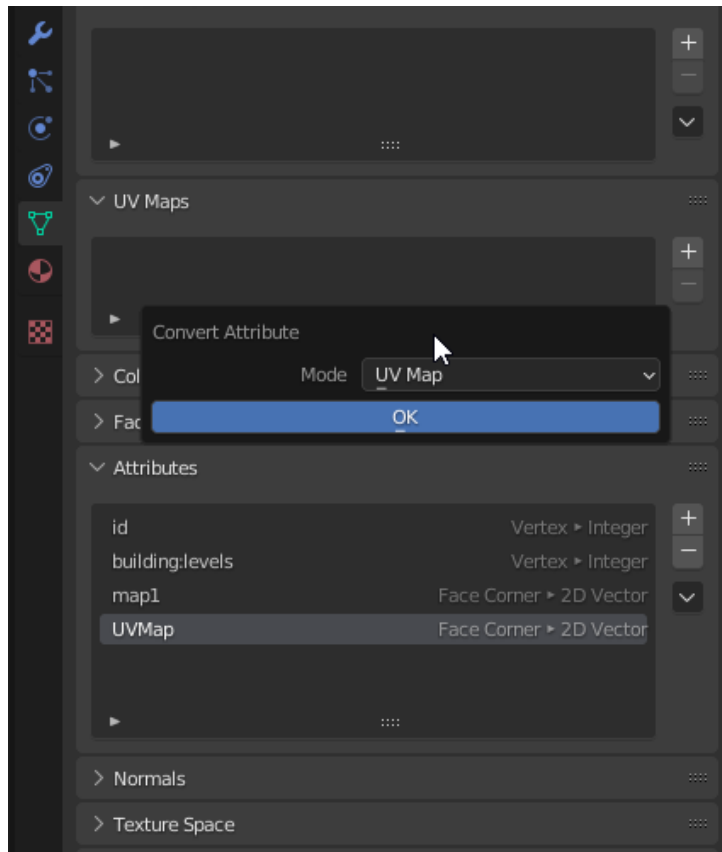
I want to use Buildify in Unreal Engine, Unity or for print. How can I export the object?

For Unreal and Unity I am testing the option to use Altermesh and Meshsync plugins to fully use the potential of object instancing. But for now there's the only old way of exporting the whole objects with duplicate meshes. To be able to convert Buildify building into a mesh, use Realize instances node at the end of your node tree (this might take a while for larger scenes especially those created with BLOSM). Then just apply the modifier by ctrl+a (with the mouse in the 3D view) > Visual geometry to mesh



Realize instance node placement

Once you apply the modifier, all UVs will disappear. That's the current Blender's limitation for working with GN. To bring UVs back, go to the mesh properties > attributes > search for UVMaP attribute (your model is using other UV name for UV maps, search that name instead, but default UVs in Blender is UVMaP) > click on dropdown arrow > Convert attribute > Mode: UV Map



Getting textures back after converting to geometry

SUPPORT OR FEEDBACK

If you have any questions or feedback feel free to join my [Discord](#) server.